Software Architecture and Design Specification

Version: 2.0 Status: Updated Draft

Revision History

Version	Date	Change Summary	Status
1.0	Earlier version	Initial architecture diagram.	Superseded
2.0	11-11-2026	Expanded architecture, component descriptions, sequence models, API contracts, security and design rationale.	Updated draft

Approvals

Role	Name	Signature / Date
Project Guide		
Team – Frontend and UX	Kundan V	
Team – Backend	Ipsa R	
Team – Data and Docs	Jai Boonlia	

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design of the Hostel Management System, including major components, their responsibilities, interactions, data storage, security controls, interfaces and design decisions.

1.2 Scope

The architecture covers browser-based student and warden portals, the Flask application, business modules, SQLAlchemy data access, relational persistence and configured external service boundaries for payment and notifications.

1.3 Audience

- Developers and maintainers
- Test engineers
- Project guide and academic evaluators
- Authorized system administrators

1.4 Definitions

Term	Meaning
Flask	Python web application framework
ORM	Object-relational mapping

RBAC	Role-based access control
API	Application programming interface
UI	User interface
DB	Database

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 describe the architecture and design respectively. The component descriptions and sequence diagrams explain the main interactions. API contracts describe logical operations; exact route paths and payload names must remain aligned with the current implementation.

2.2 Related Documents

- Hostel Management System SRS, updated version 2.0.
- Hostel Management System Software Test Plan, updated version 2.0.
- Project source repository and automated pytest suite.

3. Architecture

3.1 Architecture Goals and Constraints

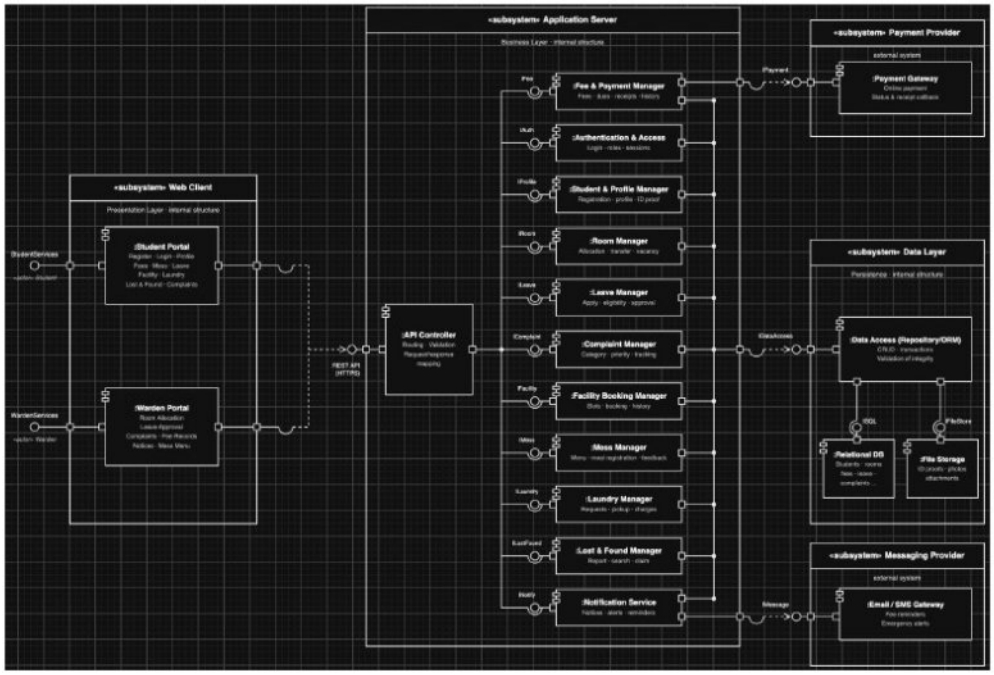
- Separate presentation, request handling/business logic and persistence responsibilities.
- Provide role-aware student and warden access.
- Support modular hostel-service functionality.
- Preserve data integrity for student, fee, room, booking and service records.
- Use the existing Flask and SQLAlchemy application stack.
- External payment and messaging services remain integration dependencies and are not assumed to be active in every environment.

3.2 Stakeholders and Concerns

Stakeholder	**Primary Concerns**
Student	Ease of use, privacy, accurate fee and room details, reliable service requests.
Warden	Correct administrative records, access control, manageable workflows and visibility.
Developer / Maintainer	Modularity, testability, maintainability, configuration and deployment.
Project Evaluator	Traceability between requirements, architecture, implementation and tests.

3.3 Component Architecture Diagram

The existing project architecture diagram is retained below as the baseline view. It shows the Web Client, Application Server/business modules, Data Layer, Payment Provider and Messaging Provider. External provider boxes represent integration boundaries and should not be interpreted as proof that a live provider connection is configured.



3.4 Component Descriptions

Component	Responsibility
Web Client / Student Portal	Presents student registration, login, profile, room, fees, facility booking, leave, mess, laundry, lost/found, complaints and notifications views.
Warden Portal	Presents warden login and authorized administrative views such as room/fee/mess and other supported records.
Authentication and Access	Validates credentials, manages session state and checks role authorization before protected operations.
API Controller / Flask Routes	Receives HTTP requests, validates request data, invokes the relevant service/model operation and returns a response or rendered page.
Student and Profile Manager	Handles student registration, profile information and student-associated records.
Room Manager	Provides room details, availability and related room information.
Fee and Payment Manager	Provides fee views, payment workflow, transaction history and receipt-related information.
Leave Manager	Processes leave submissions and status/history operations.
Facility Booking Manager	Checks facility availability and records booking/cancellation operations.
Mess Manager	Provides mess menu, meal registration, attendance, feedback, holiday and fee-related functions.
Laundry Manager	Handles laundry requests and related status/charge

	information where enabled.
Complaint Manager	Stores complaint category, priority, description, attachment reference and status.
Lost and Found Manager	Maintains lost/found item reports and claim/return information where enabled.
Notification Service	Presents or dispatches notices and reminders where configured.
Data Access Layer (SQLAlchemy ORM)	Maps application entities to relational tables and performs database operations.
Relational Database	Persists application records such as students, rooms, fees, payments, leave, bookings, mess, complaints and related data.
File Storage	Stores uploaded identity or complaint files under configured upload restrictions.
External Payment / Messaging Providers	Optional external boundaries for payment and email/SMS delivery; availability depends on configuration.

3.5 Chosen Architecture Pattern and Rationale

The system follows a layered, modular web application architecture. The browser-based presentation layer communicates with Flask routes/controllers. Application logic is organized around hostel-service modules. SQLAlchemy provides the persistence abstraction over a relational database. This approach fits the current project size and technology stack, keeps deployment straightforward and supports focused testing. A distributed microservices architecture is not selected because it would add deployment, communication and operational complexity without a demonstrated need.

3.6 Technology Stack and Data Stores

Layer	**Technology / Design**
Backend	Python with Flask
Data access	SQLAlchemy ORM
Database	Configured relational database for application use; SQLite used for automated tests
Frontend	HTML templates, CSS and browser-side interaction
Authentication	Flask session/login management and role checks
Testing	pytest and Flask Test Client
File handling	Application-managed upload directory with validation and access restrictions
External integrations	Payment and notification providers only when configured

3.7 Risks and Mitigations

Risk	**Impact**	**Mitigation**
Unauthorized role access	Exposure or modification of administrative records.	Enforce role checks on every protected route and test direct URL access.

Database outage or connection failure	Service operations may fail.	Use environment-based configuration, backups and clear failure handling.
Invalid or malicious uploads	Storage or application security risk.	Validate file type/size, use safe filenames and restrict file access.
Payment provider unavailable	Payment cannot be completed.	Handle failure states, avoid duplicate records and reconcile provider responses.
Module changes cause regression	Previously working workflows may break.	Maintain automated regression tests and requirement-to-test traceability.

3.8 Traceability to Requirements

SRS Requirements	Architecture Components
FR-001 to FR-003	Student Portal, Authentication, Student/Profile Manager, Data Access Layer
FR-004	Room Manager, Warden Portal, Data Access Layer
FR-005	Facility Booking Manager, Student Portal, Data Access Layer
FR-006 and FR-013	Fee/Payment Manager, Warden Portal, Data Access Layer, optional Payment Provider
FR-007 and FR-014	Mess Manager, Warden Portal, Data Access Layer
FR-008 to FR-011	Leave, Laundry, Complaint, Lost/Found and Notification modules
FR-012, SR-001, SR-002, SR-006	Authentication and Access component; route-level authorization

3.9 Security Architecture

- Authentication: protected operations require an authenticated session.
- Authorization: student and warden capabilities are separated using role checks.
- Password protection: passwords are stored as hashes, not plaintext.
- Input validation: server-side validation is required for form submissions and uploaded files.
- Data confidentiality: student-specific records must be scoped to the authenticated student.
- Transport security: HTTPS is required in production deployment.
- Secrets management: database credentials and secret keys must be supplied through protected environment configuration.
- Logging: do not log passwords, session secrets or sensitive payment credentials.

4. Design

4.1 Design Overview

The design is organized around request-response workflows. A browser submits a form or requests a page. Flask routes authenticate and authorize the request, validate input, call the relevant module/model operation and use SQLAlchemy to read or persist data. The response is returned as a rendered page, redirect or structured response. Database changes should be committed only after validation and should be rolled back when an operation fails.

4.2 UML Sequence Diagrams

Figure shows a student fee-payment workflow. The automated test suite includes fee-payment and payment-history behaviour; the diagram is a logical interaction model rather than a claim about a live external gateway.

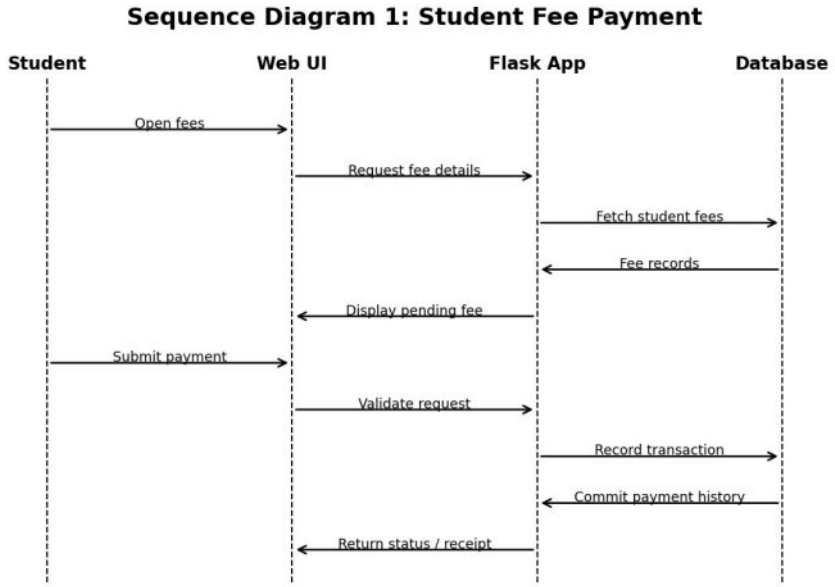
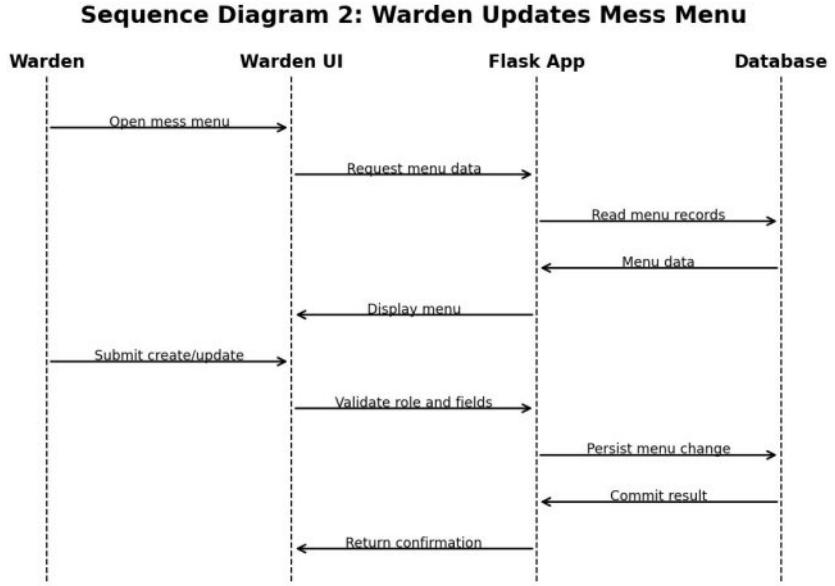


Figure shows an authorized warden creating or updating a mess menu entry. Warden authentication and mess-menu create/update/delete operations are represented in the automated test coverage.



4.3 API / Interface Design

The current application is primarily a server-rendered Flask web application. The following are logical interface contracts for its main components; route URLs and field names must match the implemented Flask routes and templates and are intentionally not fabricated here.

Interface	Method / Trigger	Input	Output / Error Behaviour
Student Authentication	POST login form	Student identifier/email	Successful authentication

		and password.	creates a session and redirects to the student dashboard; invalid credentials return an error message.
Student Registration	POST registration form	Required identity, contact, academic, parent and hostel fields; ID proof if required.	Creates a student record on valid input; returns field validation errors for invalid or missing values.
Fee Payment Workflow	POST payment action	Authenticated student context and selected fee/payment details.	Records successful payment and history/receipt information; rejects unauthorized, invalid or failed payment attempts.
Warden Mess Menu	GET and POST/PUT/DELETE-style form actions	Authenticated warden context and menu fields.	Returns menu records or confirms valid create/update/delete; rejects invalid or duplicate entries and unauthorized access.

4.4 Error Handling, Logging and Monitoring

- Validate all user input on the server; do not rely only on browser-side validation.
- Return clear user-facing messages for invalid credentials, missing fields, invalid values and unavailable resources.
- Handle database exceptions without exposing internal stack traces or credentials to users.
- Use transaction boundaries for operations that update multiple related records.
- Record operational errors in application logs while excluding passwords and sensitive data.
- Monitor application availability, database connectivity, failed requests and payment/booking errors in deployment.

4.5 UX Design

- Provide separate student and warden navigation according to role.
- Use consistent page structure, headings, form labels and action placement.
- Identify required fields and explain validation failures near the relevant field.
- Show confirmation for successful registration, booking, leave submission, payment and administrative updates.
- Use responsive layouts suitable for desktop and mobile browsers.
- Provide status labels and history views so users can understand pending and completed requests.

4.6 Open Issues and Next Steps

- Confirm the final deployed database engine and configuration for production.
- Confirm whether external payment and email/SMS providers are configured and test their sandbox/live integrations separately.
- Verify laundry, lost-and-found and notification capabilities against the final deployed build; some are included in the requirements/architecture baseline but are not fully represented by the currently inspected automated test subset.
- Create and review the two formal UML use-case diagrams against the final route and role permissions.
- Conduct browser compatibility, accessibility, performance and security testing before production release.

4.7 Modularity and Interface Contracts

The application is divided into cohesive modules around student services, warden operations, authentication, room information, fees, facility booking, mess, leave, laundry, complaints, lost and found, and notifications. Each module should expose only the operations needed by other modules and should avoid depending on another module's internal implementation.
An API is the source-level contract between software components. For this server-rendered Flask application, the practical contracts include route/controller operations, form fields, validation outcomes, model/service calls and rendered responses. The API contract should document operation purpose, input, output, errors and authorization requirement.

An ABI concerns binary-level interoperability between compiled components or external libraries. The current Python/Flask application does not define a project-specific ABI between its own modules; any ABI compatibility concern is delegated to compatible Python packages, native extensions and the deployment runtime. This distinction follows the Unit 2 discussion of API and ABI.

Interface / Contract	Purpose	Input	Output / Failure Behaviour
Student authentication	Establish a student session	Institutional login credentials	Student dashboard/session on success; authentication error on failure
Warden authentication	Establish an authorized warden session	Warden credentials	Warden dashboard/session on success; deny invalid or unauthorized access
Fee workflow	Read fee data and initiate configured payment workflow	Authenticated student context and selected fee/payment data	Updated payment record/history/receipt information when successful; validation or authorization error otherwise
Facility booking	Create a booking for an available slot	Authenticated student, facility, date and slot	Booking confirmation/history; reject unavailable or conflicting slot
Mess menu administration	Maintain menu entries	Authorized warden context and menu fields	Persisted menu entry; reject invalid or duplicate data

4.8 UML Interaction Descriptions

The following interaction descriptions are the textual sequence specifications for the two principal workflows. They are consistent with the sequence-diagram concepts in the Unit 2 slides and can be converted directly into UML sequence diagrams.

Student Fee Workflow – Lifeline	Interaction
Student	Opens the fee section and selects a pending fee.
Browser / UI	Submits the payment request with the selected fee and form data.
Flask Route / Controller	Checks the authenticated session and verifies that the fee belongs to the student.

Fee / Payment Logic	Validates the request and invokes the configured payment workflow.
SQLAlchemy / Database	Stores the payment result and updates the related fee/payment status when successful.
Flask Route / Controller	Returns a success/receipt view or an error response.
Browser / UI	Displays the resulting status and payment history.
Warden Mess Menu Workflow – Lifeline	**Interaction**
Warden	Signs in and opens mess-menu administration.
Browser / UI	Submits a create, update or delete request.
Flask Route / Controller	Checks the session and warden role.
Mess Menu Logic	Validates the menu data and checks duplicate/invalid entries.
SQLAlchemy / Database	Persists the authorized change.
Flask Route / Controller	Returns a confirmation or validation/authorization error.
Browser / UI	Displays the updated menu or error message.

4.9 Design Review and Quality Practices

- Review changes for modularity, clear responsibilities, readable code and consistency with the SRS.
- Use peer/code review to identify defects before integration; record review findings and their resolution.
- Keep interface contracts stable and update dependent modules and tests when an interface changes.
- Use automated regression tests after changes to reduce unintended effects on existing workflows.
- Track design risks such as external service availability, database configuration, access-control errors and incomplete module verification.

Architecture diagrams and UML models are conceptual views of the current Flask application. They do not imply that optional third-party payment or messaging providers are live or that every module has been independently verified in production.

5. Appendices

5.1 Glossary

HMS: Hostel Management System; ORM: Object-Relational Mapping; RBAC: Role-Based Access Control; UI: User Interface; API: Application Programming Interface.

5.2 References

- Hostel Management System SRS, updated version 2.0.
- Software Engineering Unit 3 and Unit 4 lecture slides supplied for this task.
- Flask and SQLAlchemy project implementation and pytest test suite.

5.3 Tools

Python, Flask, SQLAlchemy, pytest, Flask Test Client, relational database, browser developer tools, Git and GitHub.